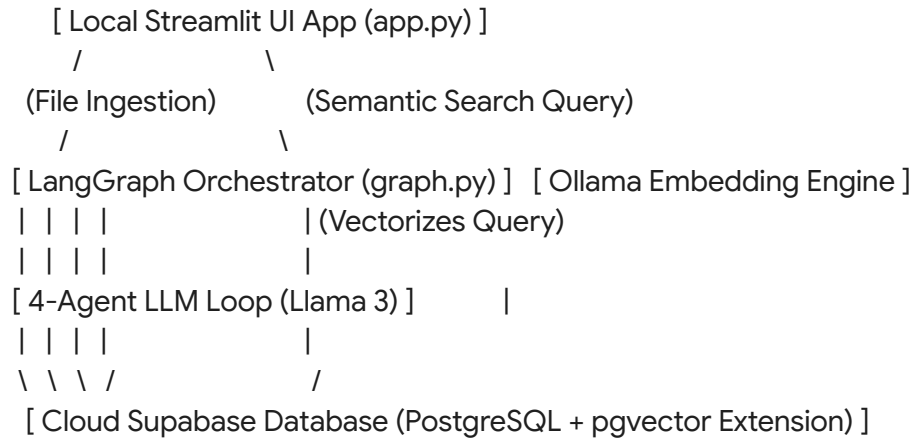
Here is the comprehensive, end-to-end Technical Architecture and Workflow Report for your **Agentic AI Faculty Hiring & Screening System**. This documentation maps out the entire lifecycle of what you have built, from the structural database design to the local multi-agent state graph machine.

🏗️ Architectural Overview & Core Infrastructure

The system is engineered as a hybrid local-cloud infrastructure designed to process unstructured resume documents, evaluate them against strict institutional compliance constraints, score their research output using weighted algorithms, and expose them to semantic natural language querying.



1. Local Inference Engine & Embedding Infrastructure

- **LLM Core:** Ollama running an open-source **Llama 3 (8B Parameter)** model locally on native Apple Silicon hardware memory.
- **Deterministic Formatting:** The LLM client is instantiated utilizing an explicit structured formatting rule (format="json", temperature=0) to force production-grade, predictable JSON schemas out of unstructured prose.
- **Vector Embeddings:** Ingestion and retrieval pathways leverage OllamaEmbeddings(model="llama3") to project raw data chunks into a high-dimensional **4,096-dimension vector space**.

2. Cloud Storage & Vector Database Layer

- **Database Engine:** Hosted **Supabase (PostgreSQL)** database.
- **Vector Engine Expansion:** Enabled the pgvector database extension natively inside the schema to support high-performance spatial geometric computations.

- **Security & Session Management:** Decoupled environmental key routing using a local secure .env file containing system parameters (SUPABASE_URL and the database-level anon public gateway passport key), injected seamlessly at runtime using python-dotenv.

🗃️ Database Schema Definition (faculty_profiles)

You provisioned and structured a custom relational SQL table designed to evolve alongside the candidate's journey through the agent loop. Below is the exact data layout matrix now handling your production data:

Column Physical Name	SQL Data Type	Key Type / Constraint	Functional Purpose
id	uuid	**Primary Key** (Auto-Gen)	Unique system tracking token for each applicant record.
created_at	timestamptz	Default: now()	Chronological transaction timestamp logging insertion.
name	text	Standard Field	Core identification string of the applicant.
email	text	**Unique Constraint**	Electronic mail address used to enforce system-wide **Upsert** constraints. Prevents concurrency duplicates.
contact	text	Standard Field	Telecom communication address string.
degrees	text[]	Array String Field	Collection of structured academic

			credentials parsed out of prose.
universities	text[]	Array String Field	Collection of educational institutions attended by the candidate.
specialization	text	Standard Field	Primary area of technical core competency.
teaching_years	integer	Metric Integer	Explicit duration count spent in academic education environments.
industry_years	integer	Metric Integer	Explicit duration count spent in corporate or production environments.
admin_roles	text[]	Array String Field	Historical leadership designations held (e.g., Head of Department).
publications_count	integer	Research Integer	Raw metric tracking total published papers.
scopus_count	integer	Research Integer	Specific index tracking Scopus journal additions.

sci_count	integer	Research Integer	Specific index tracking Science Citation Index records.
citations	integer	Research Integer	Total tracking index for intellectual reach volume.
h_index	integer	Research Integer	Mathematical metric balancing publication count vs citation weights.
patents	integer	Research Integer	Volume tracking for legal industrial innovations held.
grants_value_inr	integer	Research Integer	Fiscal value metric tracing financial backing received in INR.
eligibility_status	text	Compliance Output	Conditional state string tracking gate verification checks (Eligible / Not Eligible).
eligibility_reason	text	Compliance Output	Human-readable textual audit logging the cause of a pass/fail verdict.
research_score	float4	Analytical Output	Aggregated algorithmic grade output from the weighting matrix.

research_breakdown	jsonb	Structured JSON Object	Deep binary key-value mapping showing exact fractional score contributions.
teaching_score	float4	Analytical Output	Aggregated score combining baseline experience and leadership weights.
final_evaluation_report	text	LLM Output Prose	Formal 4-line summary compilation report written by the final coordinator agent.
resume_embedding	vector(4096)	**Vector Matrix Field**	High-dimensional semantic mathematical coordinates mapping the holistic profile meaning.

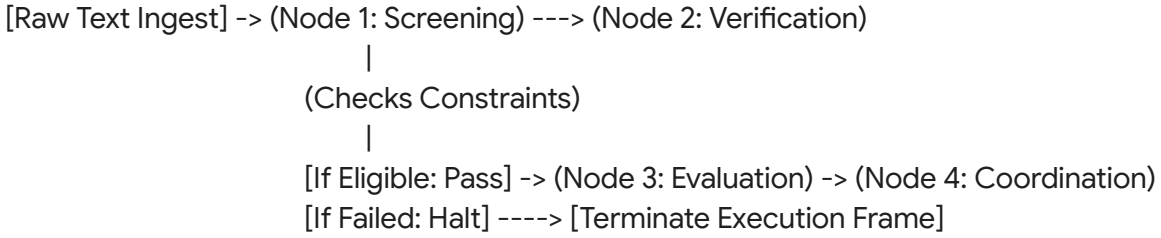
🔄 End-to-End Execution Workflow Lifecycle

Phase 1: Ingestion and Pre-Processing (app.py)

1. The user launches the application via the local host terminal utilizing python3 -m streamlit run app.py.
2. The user drops a candidate resume file (.pdf or .docx) into the Streamlit drag-and-drop web file component widget.
3. The underlying ingestion scripts leverage custom utilities (such as PdfReader) to strip raw bytes down to basic unformatted python string characters.
4. The system updates the global LangGraph runtime memory wrapper (FacultyState) with the raw text, combined with the target institutional job boundaries (e.g., requiring a Ph.D., minimum 3 teaching years, and a "Computer Science" specialization core focus).

Phase 2: The Multi-Agent Orchestration Graph Loop (graph.py)

Once the user executes the pipeline via the user interface button, control is securely handed over to a custom compiled **LangGraph State Machine Architecture**. The machine executes four sequential processing phases:



Node 1: Application Screening Agent

- **Operation:** The raw, messy textual string is packed into an un-tuned structural parser instruction block. Local Llama 3 processes the request and executes structural data extraction into a pure JSON schema.
- **Vector Engine Invocation:** Concurrently, the text is dispatched to the local vector engine, which calculates its 4,096 dimensional float coordinates mapping the candidate's exact background concept.
- **Database Synced Pipeline (The Upsert Guard):** The system initiates an asynchronous cloud connection. Utilizing the .upsert(db_payload, on_conflict="email") directive, the database scans for an existing record matching the candidate's email. If found, it safely overwrites the structural metrics to eliminate table clutter; otherwise, it appends a clean record row.

Node 2: Qualification Verification Agent

- **Operation:** A deterministic programmatic check extracts the academic array strings and checks them against institutional gate rules.
- **Logic Matrix Evaluated:**
  - *Rule A:* Checks for the explicit phrase substring "phd" or "ph.d" within the candidate's degrees.
  - *Rule B:* Compares the candidate's teaching_years integer directly against the required minimum tracking limit.
  - *Rule C:* Evaluates if the required specialized department target maps directly into the candidate's academic specialization text block.
- **Cloud Logging Update:** The agent locks down a compliance status variable (Eligible or Not Eligible), drafts an explicit reason code string, updates the live cloud row matching the active UUID, and dynamically manages the graph trajectory. If the candidate fails a

gate check, the conditional edge forces an immediate graph execution halt (END) to conserve local computing power.

Node 3: Research & Teaching Evaluation Agent

- **Operation:** If the candidate passes Phase 2, they enter the algorithmic scoring chamber. This agent calculates structural fractional mathematical weights to score the candidate's entire research output portfolio out of 100 points:
  $$Publications Score (30\%) = \min\left(\frac{\text{Publications}}{10} \times 100, 100\right) \times 0.30$$
  $$Citations Score (25\%) = \min\left(\frac{\text{Citations}}{100} \times 100, 100\right) \times 0.25$$
  $$H\text{-}Index Score (20\%) = \min\left(\frac{H\text{-}Index}{5} \times 100, 100\right) \times 0.20$$
  $$Patents Score (10\%) = \min\left(\frac{\text{Patents}}{2} \times 100, 100\right) \times 0.10$$
  $$Grants Score (15\%) = \min\left(\frac{\text{Grants (INR)}}{1,000,000} \times 100, 100\right) \times 0.15$$
- **Database Cloud Append:** The fractional breakdown dictionary along with the unified floating-point score is written to the cloud Postgres row using an isolated .update() network packet query.

Node 4: Interview Coordination Agent

- **Operation:** Takes the raw structured score outputs and evaluates the historical experience arrays to formulate an overarching score for teaching capabilities.
- **Synthesis generation:** The cumulative scores, alongside candidate metrics, are packaged into a generative prompt. Local Llama 3 writes a precise, formal 4-line executive summary assessment report containing a clear final recommendation for the hiring panel.
- **Final Cloud Row Lock:** The summary text block is written into the final_evaluation_report cell of the live matching candidate row, bringing the LangGraph loop execution frame to a successful, clean termination (END).

Phase 3: The Semantic Talent Search Pipeline

Once files are safely locked inside the database with their respective high-dimensional vector signatures, the application exposes an on-demand, instant analytical querying mechanism that completely bypasses file uploading entirely:

1. The recruiter opens the system and inputs a standard human-language search intent phrase into the second UI window tab (e.g., *"Show me computer science faculty with high research citation metrics"*).
2. The search string is instantly converted by the local model into an active **4,096-dimensional query vector array**.
3. The local client triggers a secure Remote Procedure Call (**RPC**) to a highly specialized custom SQL function stored natively inside the Supabase server database

(match_resume_embeddings).

4. **Cosine Similarity Space Math:** The database utilizes a highly optimized native geometric mathematical operator (<=>) to evaluate the inverted cosine similarity angle between your active query array and the static array values stored in the rows:

$$\text{Similarity} = 1 - (\text{resume\_embedding} \Leftrightarrow \text{query\_embedding})$$

5. The cloud database ranks all existing records by their mathematical similarity score, filters out rows falling below a 30% match threshold, caps the output query space to the top 3 closest profile logs, and sends them back down the network link.
6. The Streamlit interface expands the matching candidate matrices dynamically on the screen, showing the exact match confidence percentage along with the AI coordination agent's executive summary.

🛠️ Key Technical Operations & Optimization Matrix

During development, you successfully resolved and implemented critical software optimization strategies required for operational stability:

- **Environment Variable Siloing:** Solved runtime race conditions by moving load_dotenv() to the absolute root level execution line of graph.py. This fixed variable loading sequence bugs caused by Python sequentially resolving file imports before initialization could finish.
- **Network Credential Realignment:** Discovered and fixed a critical connection issue (Error 401) by separating frontend routing keys (sb_publishable_...) from heavy-duty, core relational database-level passports (anon / public tokens beginning with eyJhbG...), which are required for secure database connection initialization.
- **Data Concurrency Protection (Upsert Matrix):** Overcame data fragmentation issues by introducing a structured SQL database tracking limit (unique_email UNIQUE (email)) combined with a .upsert() command line override logic statement. This guarantees that your data architecture stays perfectly organized and free of duplicates regardless of user re-trigger counts.